

Formal Methods in Software Engineering

Florentin Ipate

Formal Methods

- Mathematically rigorous techniques for the specification, development and verification of software and hardware systems.

Formal Methods – applications

- Specification/Modelling
 - Finite State Machines (FSM)
 - Extended FSM (EFSM)
 - Natural computing models (membrane systems)
- Formal verification
 - Proofs
 - Model checking
- Testing
 - Model-based testing
 - Search based software testing

Course Topics

- Modelling, analysis and model checking with Event-B, Rodin and ProB
- Model-based testing (extended FSM / Stream X-Machine based)
- Search-based Testing
- Formal methods for testing and verification using the Haskell language
- Using Membrane Systems (P Systems) for modelling, formal verification and testing of systems

Finite State Machines (FSMs) - core concept

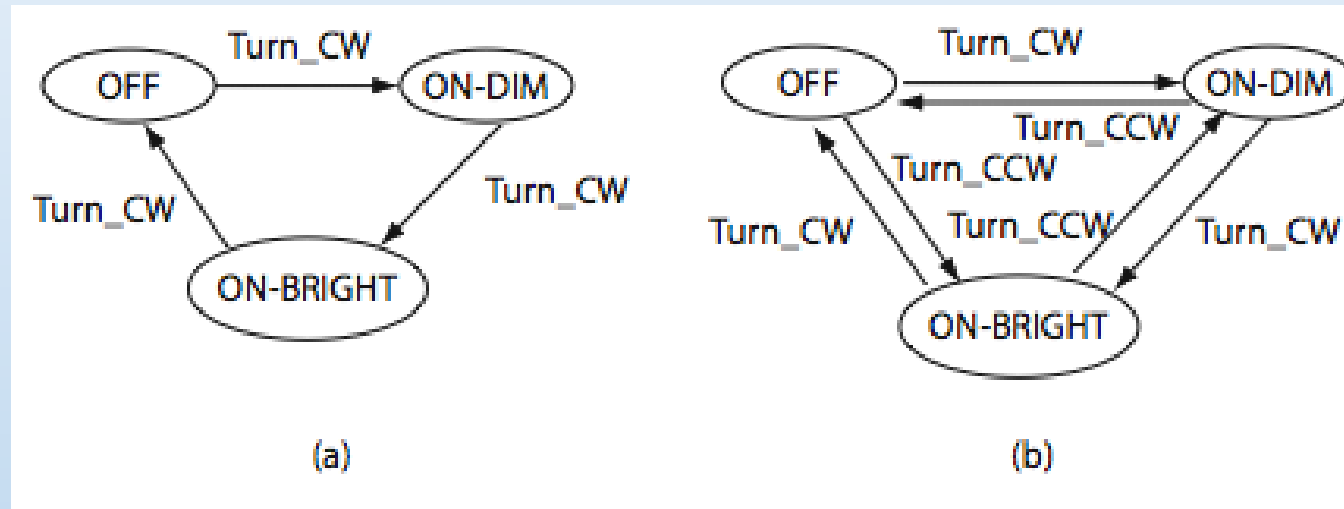
- An abstract machine that can be in exactly one of a finite number of states at any given time.
- It changes from one state to another in response to some inputs (events).

FSM - structure

- **States:** The specific conditions the system can be in.
- **Events (Inputs):** Triggers that cause a reaction.
- **Transitions:** The movement from one state to another based on an event.
- **Actions:** Calculations or side effects that happen during a transition.

FSM: lamp example

Simple three state lamp behavior:



(a) Lamp switch can be turned clockwise.

(b) Lamp switch can be turned clockwise and counterclockwise.

Extended FSM

- An FSM always performs a transition on a given input from one state to another.
- An EFSM, however, only performs a transition when a given set of conditions has been satisfied.

Extended FSM

- An EFSM allows you to use variables; for example, you could implement a for-loop.
- This allows you to model algorithms that would otherwise require a combinatorial explosion of states.
- A Stream X-Machine is a type of EFSM.

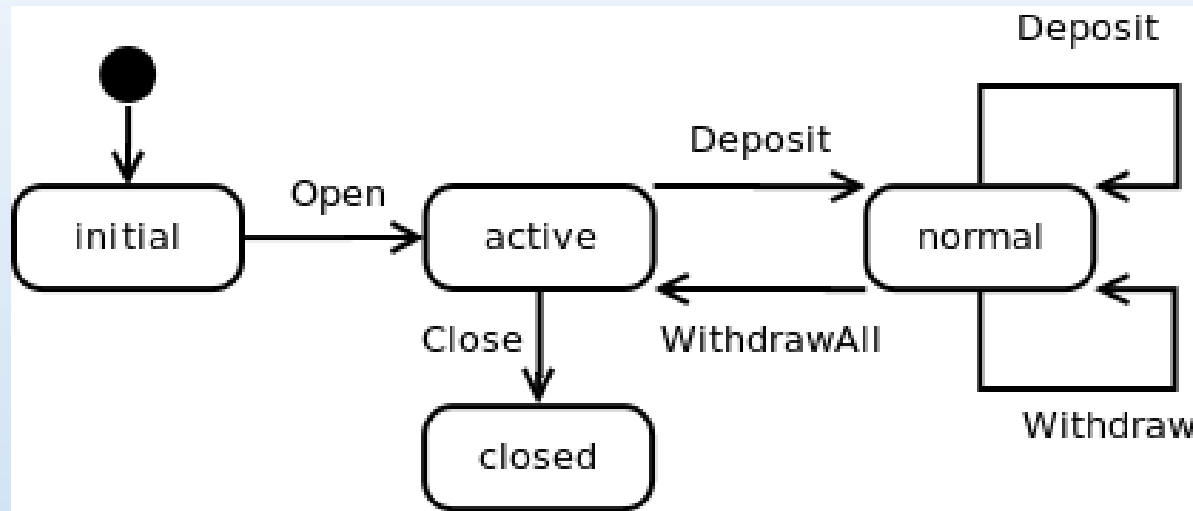
Stream X-Machines

- A Stream X-Machine (SXM) is like a FA with the following differences:
 - the machine has **memory**
 - transitions are labelled with **processing functions** which
 - operate on inputs and memory,
 - update the memory and
 - produce outputs

Processing functions

- A processing function consists of:
 - A **guard** condition depending on the input and the memory
 - An **effect** which
 - changes the memory and
 - produces an output

SXM Account



$\text{Open}(b, \text{open}()) = (\text{openOut}, 0)$

$\text{Deposit}(b, \text{deposit}(a)) = (\text{depositOut}, b + a)$

$\text{Withdraw}(b, \text{withdraw}(a)) = (\text{withdrawOut}, b - a)$ if $a < b$

$\text{WithdrawAll}(b, \text{withdraw}(a)) = (\text{withdrawOut}, 0)$ if $a = b$

$\text{Close}(b, \text{close}()) = (\text{closeOut}, b)$

Initial Memory
 $b = 0$

Membrane systems/ P systems

- What is a P System?
- A distributed, parallel computing model introduced by Gheorghe Păun (1998) inspired by the structure and functioning of living cells. Instead of bits and bytes, it computes using "chemicals" inside "membranes."

Cell-like P systems

- **The Structural Model** Imagine a hierarchical arrangement of compartments, similar to a cell within a cell.
- **The Skin:** The outermost membrane separating the system from the environment.
- **Regions:** The spaces inside membranes.
- **Objects:** The data. Unlike standard arrays, these are **Multisets** (e.g., {a, a, b, c}). Order doesn't matter, only the quantity of atoms.

Cell-like P system computation

- How it Computes (The Rules)
- Each region contains evolution rules that act on the objects:
- Transformation: object a becomes objects b and c ($a \rightarrow bc$).
- Communication: move object a into the inner membrane ($a \rightarrow \text{in}$) or send it out ($a \rightarrow \text{out}$).
- Dissolution: a membrane can dissolve, releasing its contents into the parent region.

Cell-like P system investigations

- **Why do we study them?**
- **Turing Completeness:** Mathematically, they are as powerful as any modern computer (Turing Machine).
- **Solving Hard Problems:** Theoretically, the massive parallelism allows them to solve NP-Complete problems (like the Traveling Salesman Problem) in polynomial time by trading **space** (creating exponential membranes) for **time**.

Cell-like P system applications

- **Real-World Applications**
- **Systems Biology:** Perfect for modeling photosynthesis, signaling pathways, and bacteria colony behavior.
- **Ecological Modeling:** Simulating interactions in an ecosystem.
- **Membrane Algorithms:** A niche in evolutionary computing used for complex optimization tasks.

Formal verification

- The process of mathematically checking that the behaviour of a system, described using a formal model, satisfies a given property (also described formally).
- Hoare logic – correctness proofs
- Refinement-based development
- Model checking – checks whether a finite-state model of a system meets a given specification (typically LTL or CTL formulae).

Software Testing

- Software testing refers to the process of executing an application or program with the intent of detecting potential software bugs.
- It also has the objective of helping the developers to find out whether the software is working according to the intended purpose.
- During the testing process, the end user's area of application will be considered and the software will be subjected to a series of tests to ensure it satisfies the needs of the end users.

Testing vs. Formal Verification

- Techniques for software validation
- *Testing* - makes sure the code actually works the way it's supposed to work.
- *Formal verification* - mathematically proves that the underlying algorithm is correct.

Testing vs. Formal Verification

- Dijkstra: “Program testing can be used to show the presence of bugs, but never to show their absence!”
- However, testing does not require a formal specification (model).
- Formal verification – used in particular for safety critical systems
- When a model exists => Model Based Testing
- Model Based Testing => Test automation

Test generation

- Exhaustive testing is usually not possible
- Test generation is a major part of software testing
 - Test cases: scenarios to be checked
 - Test data (suites): actual values used in testing
- Automated test execution is relatively easy
- Automated test generation is much more complicated
- The existence of a formal model helps a great deal

Model-Based Testing

- Typically state-transition models are used (FSM, EFSM, Statecharts)
- Test generation can be automated
- Various coverage criteria are used
- More powerful FSM based methods (e.g. W-method) can even guarantee some kind of fault detection

Search Based Software Testing

- The use of a meta-heuristic optimizing search technique to automate a testing task, e.g. the automatic generation of test data.
- Used when the search space is very large and the problem to be solved is complex.
- Optimizing search technique:
 - Local, e.g. hill-climbing, simulated annealing
 - Global, e.g. genetic algorithms
- Key to the optimization process is a problem-specific fitness function - guides the search to good solutions within a practical time limit.